MVP

One of the simplest requirements for our MVP (and final product) is that it is both written natively in Rhino and compiles cleanly into native Rhino. Which also means it has to be its own standalone executable that works with the RhinoCommon software. The overall flow of the project is also expected to follow the prototype that the sponsor provided, albeit that one was in Grasshopper.

The program should be able to accept an Excel/CSV/JSON spreadsheet of the rooms and parse it. Information such as room numbers, areas, departments, and adjacencies should all be present in the provided spreadsheet. The sponsor has suggested having a default template to use. The data from the template should be accessible to the program and to Rhino.

One other requirement given by the sponsor is to include the use of a bubble diagram for adjacencies and room spacing. The bubbles will be connected by lines that represent required adjacencies. Each bubble will have a color that corresponds to its department. Each bubble's size will correlate with their area, meaning bigger rooms will have bigger bubbles. None of the bubbles should overlap each other. Information for each room, such as its department number, name, and room number, will be displayed in its corresponding bubble. We are to have a set number of iterations for creating the bubble diagram in order to account for other possible choices, rather than just taking the first diagram produced.

Rooms will then be clustered together by departments and then laid out onto their given floors, leaving some spaces between clusters in order to account for corridor placement. Corridors, vertical corridors (such as stairs and elevators), and exits will be placed after rooms are laid out. Laying out the rooms and clusters will also be iterable in order to optimize the usage of space. If not all rooms can fit on their assigned floor, the layout must be using as much space as possible to make as many rooms fit as it can while still having space available for corridors.

A scoring mechanism is required in order to properly compare layouts. Things like egress distance, adjacency satisfaction, grid alignment, area deviations, and space usage will all be used to rank a current layout, determine how well the given space is being used, and identify where it could be improved. These metrics will be displayed for the user to see to serve as explanation for why the proposed layouts were chosen. This can also allow the user to later tweak the layout if they would prefer to prioritize certain metrics over others.

The sponsor has repeatedly and strongly put emphasis on whatever layout(s) that the program suggests need to be backed up with hard evidence as to why the program selected those layouts. Any diagram we show has to show the various metrics that led to that one being picked, and anything else like that. It is incredibly important for the architect(s) to know why something was selected, as they potentially have different priorities or need to balance different things, instead of just blindly trusting the program.

Another requirement is to take the layout(s) that the program determines are a good fit and turn them back into 3D models that Rhino can display, for the architect(s) to get a proper visual representation of the choices they're making. They can also then tweak the layout much more easily. A sub-requirement of this is that the individual room's models should have the metadata attached to them that indicates what room they are, what section they are, etc.

The floors, for multi-floor buildings, also need to have a similar envelope to each other. That is, the external perimeter needs to be roughly, if not exactly, the same shape as the other floors, to minimize dead space.

And then, for stretch goals. We have a couple. A relatively plausible one is, when we model the rooms and floors back in Rhino, to potentially lay out a guess at what the furnishings of the room would look like. Like chair and desk placement for a classroom, or cubicles for an office space.

Following that, another stretch goal is to have a selector for the type of a building. Like a school, a hospital, an office space, etc., and then enforcing the varying requirements that those kinds of spaces require.

Lastly, another stretch goal is to have a PDF export of the output of our program, presenting several different layouts and the various metrics and reasoning behind them, as well as then having easily shareable results.

Research

Aiden Sperr's Research:

My focus is the solver portion of the project, which mainly encapsulates the bubble solver, but also touches floor assignment and the layout stages. At its core, this is the Quadratic Assignment Problem: to minimize $\sum f_{ij} \cdot d(p_i, p_j)$ over room placements. QAP is NP-hard, so I must depend on heuristics to assist us with the problem.

The prototype's bubble solver uses relaxation on one large cloud of circles. Methods such as Eades' spring embedder (1984) and Fruchterman and Reingold (1991), model rooms as bodies (or circles), with the required adjacencies acting like springs and the overlap of two bodies acting with a repulsive force. It then iterates these simulations until the system settles down. They are fast and give a good sense of proximity.

However, these layouts can still leave adjacency edges crossing when there may be a crossing-free arrangement. If the graph is non-planar, some adjacencies cannot all be honored. To address this, I plan to apply a graph untangling technique inspired by Goaoc et al. (2009) after the relaxation of the bubble solver. The method is to count edge crossings, then apply local moves such as swapping or relocating bubbles to reduce them, using the crossing count as a penalty in the objective (i.e., if the crossing count increases, generally, it is not a 'good movement'). A crossing-free embedding is also a prerequisite for constructing a rectangular dual of the adjacency graph (Kozminski & Kinnen, 1985), which is how bubble positions can later become rectangles, and further on, rooms in Rhino.

The floor assignment aspect resembles graph partitioning. The prototype utilizes weighted k-means, which clusters by position rather than by relationships. Instead, I propose minimizing the weight of adjacencies cut between floors while respecting each floor's area capacity, so departments that work together stay together.

In addition, the prototype's row packing is greedy, meaning that nothing is ever reconsidered or scored. Simulated annealing (Kirkpatrick et al., 1983) addresses this by perturbing a layout. This involves swapping rooms or moving rooms between floors and accepting worse moves with decreasing probability to escape local minima.

Because the scorer is separate from the solver, these methods can be compared directly on net-to-gross ratio, weighted travel distance, adjacency satisfaction, and area error. The

pipeline I am currently considering is a force-directed seed, then the partition-based floor assignment, rectangular tiling (e.g., squarified treemaps, Bruls et al., 2000), and annealing refinement. This would be multithreaded with deterministic seeding. In addition, I would implement benchmarks to grade or review the efficacy of the new methods implemented.

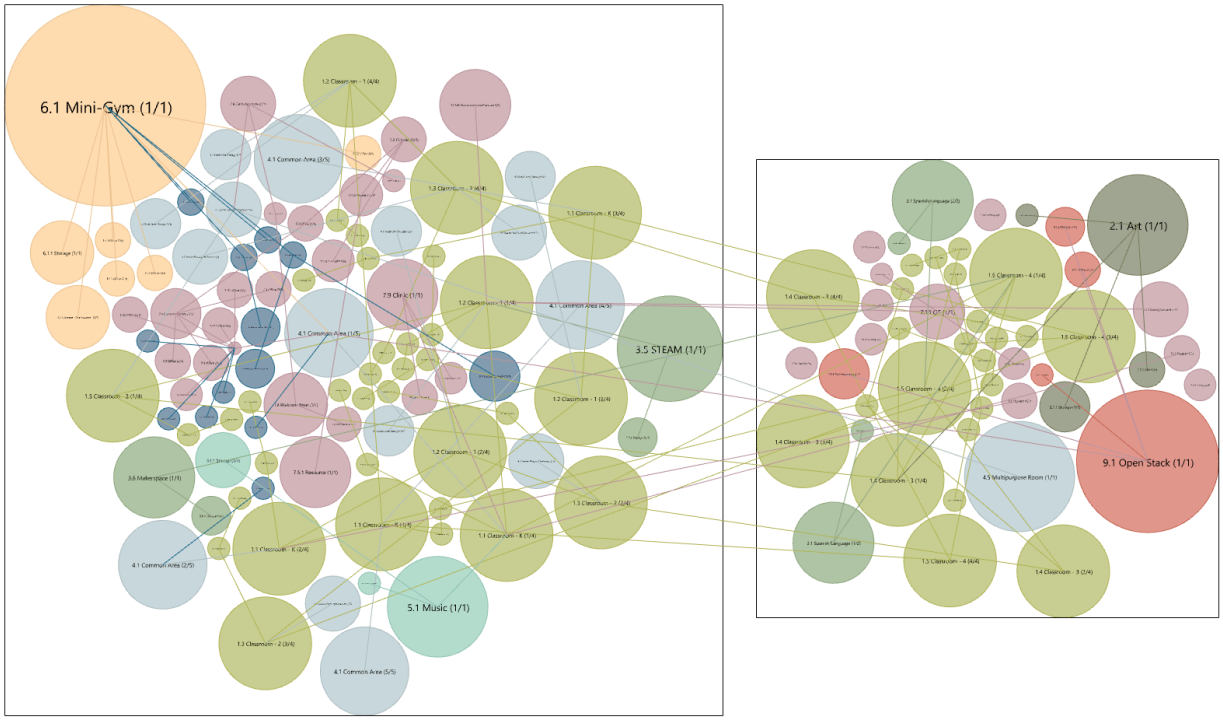
Bubble Solver from the Prototype

Once the bubble solver loads in all the data from the excel sheet, we can begin the process of making a bubble diagram from the excel spreadsheet.

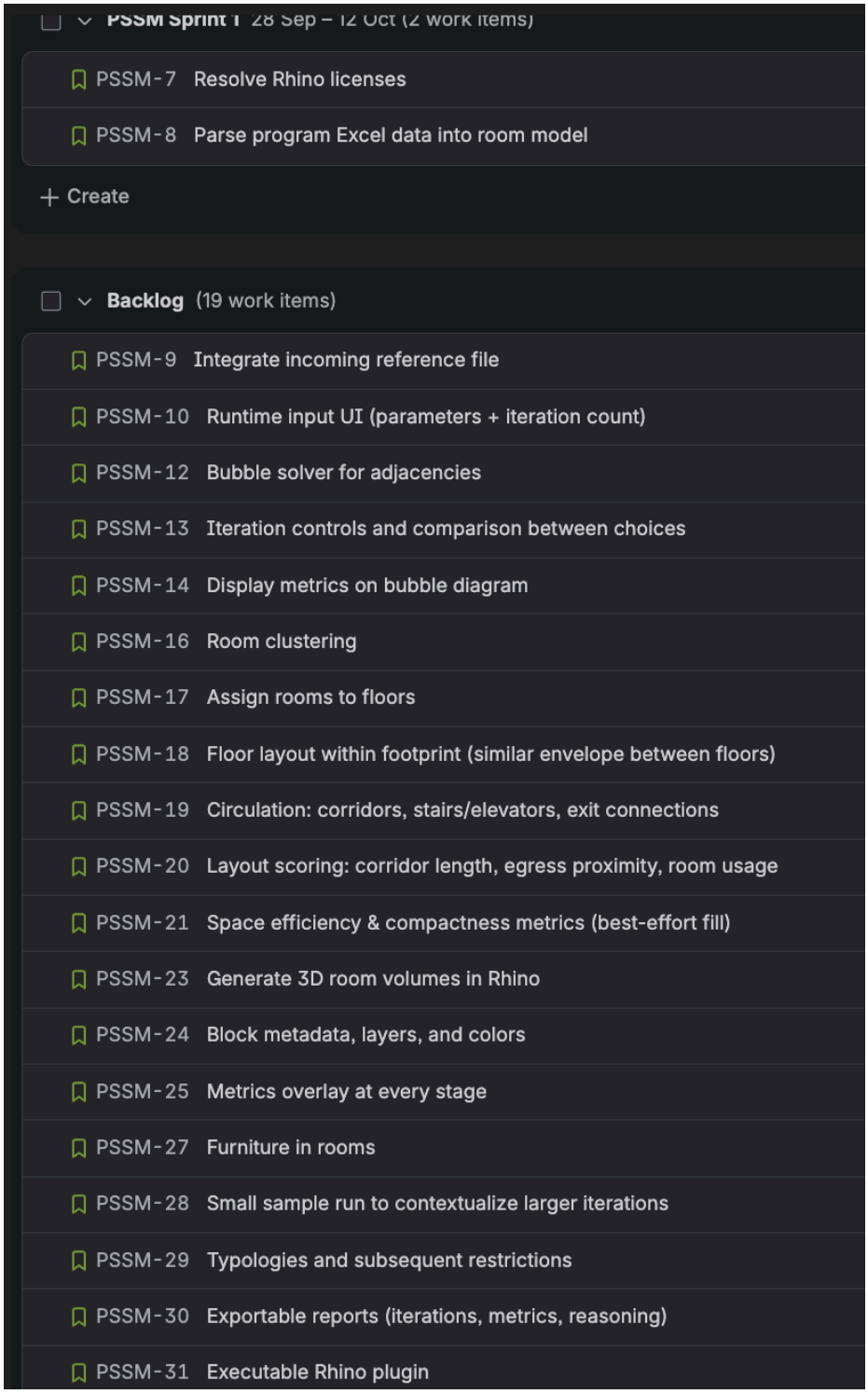
1. The first we take a rooms area (ft^2) and convert it into Rhino's unit system, then we convert that into a circular representation, finding what the radius of the equivalent area is.
2. Next, we randomly place rooms in a ring around each floor's center point.
3. We then apply 3 forces to each room for N number of iterations (each room has capped movement per step so a strong force can't launch them too far):
    1. Spring force
        1. Pulls adjacent rooms towards each other.
    2. Repulsion force
        1. Pushes overlapping rooms away from each other. This only affects rooms on the same floor, not on different floors. So, if we have 2 rooms, they will only be pushed away from each other if they are on the same floor AND are overlapping with one another.
    3. Floor cohesion
        1. Keeps the floor's rooms "squished" together so they don't drift apart.
4. Afterwards we do a run-through of the rooms, checking if any are overlapping. If so, we apply the repulsion force, so they don't overlap anymore.
5. Recenter each floor to their local origin. Then, based on their view, we either stack them on top of each other(vertically) or put next to each other(horizontally)

6. Finally, we package everything in the Grasshopper DataTree per floor followed by a string with information on the rooms, edges, floors, system units, view, spring force, repulsion force, floor cohesion and recentering.

At the end the output should look like this:



Sprint Backlog


PSSM Sprint 1 28 Sep – 12 Oct (2 work items)
PSSM-7 Resolve Rhino licenses
PSSM-8 Parse program Excel data into room model
+ Create
Backlog (19 work items)
PSSM-9 Integrate incoming reference file
PSSM-10 Runtime input UI (parameters + iteration count)
PSSM-12 Bubble solver for adjacencies
PSSM-13 Iteration controls and comparison between choices
PSSM-14 Display metrics on bubble diagram
PSSM-16 Room clustering
PSSM-17 Assign rooms to floors
PSSM-18 Floor layout within footprint (similar envelope between floors)
PSSM-19 Circulation: corridors, stairs/elevators, exit connections
PSSM-20 Layout scoring: corridor length, egress proximity, room usage
PSSM-21 Space efficiency & compactness metrics (best-effort fill)
PSSM-23 Generate 3D room volumes in Rhino
PSSM-24 Block metadata, layers, and colors
PSSM-25 Metrics overlay at every stage
PSSM-27 Furniture in rooms
PSSM-28 Small sample run to contextualize larger iterations
PSSM-29 Typologies and subsequent restrictions
PSSM-30 Exportable reports (iterations, metrics, reasoning)
PSSM-31 Executable Rhino plugin